

Prototipo funcional del chat empresarial cliente-servidor en Go

Proyecto Integrador, segundo entregable (semana 10)

Programación Orientada a Objetos

Cesar Arciniegas, Ariel Guerrero y Jorge Aguilar

UIDE

Docentes: Iván Galo Reyes Chacón y María del Carmen Salazar Torres

09/11/2026

1. Nombre del sistema seleccionado

El sistema seleccionado es el chat empresarial entre dos sedes, que corresponde a la opción B de las aplicaciones propuestas en el proyecto integrador (Universidad Internacional del Ecuador [UIDE], 2026a). En la planeación de la Etapa 1 lo describimos como una herramienta interna para una empresa de soluciones tecnológicas que acaba de abrir una segunda sede, y este prototipo es su primera versión ejecutable (UIDE, 2026b).

2. Objetivo del prototipo

El objetivo es construir una primera versión ejecutable del chat, en Go y con arquitectura cliente-servidor, que permita identificarse, enviar y recibir mensajes y consultar el historial. Con ella aplicamos estructuras, constructores, métodos, encapsulamiento, composición, validación de datos y manejo explícito de errores (UIDE, 2026b). No pretende ser el producto final: la autenticación real, el cifrado y la integración entre sedes llegan en las semanas 11 y 12.

3. Funcionalidades implementadas

La Tabla 1 resume lo que el prototipo ya hace, con el comando del cliente que lo activa y el caso de uso y el requisito de la Etapa 1 al que corresponde (UIDE, 2026b).

Tabla 1

Funcionalidades implementadas y su relación con los casos de uso

Funcionalidad	**Comando del cliente**	**Caso de uso**	**Requisito**
Identificarse e iniciar sesión	/login usuario	CU-01 (parcial)	RF-02 (parcial)
Cerrar sesión	/logout y /quit	CU-02	RF-03
Enviar un mensaje a una sala	/say, o texto después de /room	CU-03	RF-05
Enviar un mensaje directo	/dm	CU-04	RF-06
Recibir mensajes en tiempo real	Automático	CU-03 y CU-04	RF-07

Consultar el historial de una sala	/history	CU-05	RF-08
Consultar el historial directo	/dmhistory	CU-05	RF-08
Ver las salas y unirse a una	/rooms y /join	CU-11	RF-09 (parcial)
Ver los usuarios conectados	/who	CU-06	RF-10
Registrar eventos	Automático, en la consola del servidor	No aplica	RF-13 (parcial)

El servidor no confía en lo que le llega del cliente: valida que el mensaje no esté vacío ni supere los 1000 caracteres, que el usuario y la sala existan, que la cuenta esté activa, que la persona pertenezca a la sala y que la solicitud sea un JSON con un comando conocido. Es la regla básica de validar en el servidor todo dato que viene de fuera (OWASP Foundation, s. f.).

4. Requisitos atendidos y pendientes

La Tabla 2 compara el prototipo con los requisitos aprobados en la Etapa 1. Marcamos como atendido solo lo que se puede demostrar con el código y con pruebas, como pide la guía de la entrega (UIDE, 2026b).

Tabla 2

Requisitos atendidos y pendientes

Requisito	**Estado**	**Cómo se atiende o qué falta**
RF-01 Crear cuentas	Pendiente	Las cuentas se cargan desde users.json; su gestión llegará con el rol de administrador
RF-02 Inicio de sesión con token	Parcial	Identifica al usuario; la contraseña y el token los aportará el servicio FastAPI mediante la interfaz Authenticator
RF-03 Cierre de sesión	Atendido	/logout; además la sesión se libera si el cliente se desconecta
RF-04 Vencimiento y control de sesiones	Pendiente	Se hará junto con el servicio de autenticación
RF-05 Mensaje a una sala	Atendido	ChatService.SendToRoom con validaciones

RF-06 Mensaje directo	Atendido	ChatService.SendDirect con validaciones
RF-07 Entrega en tiempo real	Atendido	El servidor reenvía el mensaje a las conexiones de los destinatarios
RF-08 Historial paginado	Parcial	Devuelve los últimos n mensajes (máximo 100); falta paginar hacia atrás
RF-09 Gestión de salas	Parcial	Se puede listar y unirse; crear y archivar quedan pendientes
RF-10 Usuarios conectados	Atendido	/who
RF-11 Estadísticas	Pendiente	Semanas 11 y 12
RF-12 Activar cuentas y asignar roles	Pendiente	Depende de la gestión de usuarios
RF-13 Registro de eventos	Parcial	El servidor escribe los eventos por consola; todavía no se guardan
RNF-01 Seguridad (hash, tokens, TLS)	Pendiente	Corresponde a FastAPI y a TLS
RNF-02 Rendimiento con 20 usuarios	Pendiente	Prueba de carga de la fase de pruebas
RNF-03 Reconexión automática	Pendiente	Hoy el cliente avisa que se perdió la conexión y termina
RNF-04 Mantenibilidad	Atendido	Paquetes por responsabilidad y 25 pruebas automatizadas
RNF-05 Portabilidad	Parcial	Probado en Linux
RNF-06 Mensajes de error claros	Atendido	Cada error explica qué falló
RNF-07 Auditoría	Parcial	Registro por consola sin el texto de los mensajes
RNF-08 Integridad	Atendido	La hora la asigna el servidor y los mensajes no se modifican

Hicimos dos cambios respecto al diseño aprobado y los justificamos en la Tabla 3, como exige la guía cuando un requisito o una decisión se modifica (UIDE, 2026b).

Tabla 3

Cambios respecto al diseño de la Etapa 1

Decisión	**Cambio**	**Justificación**
Transporte	De WebSocket a TCP con un mensaje JSON por línea	WebSocket exige una librería externa. Con TCP el prototipo se

		ejecuta con solo Go y conserva lo esencial: una conexión persistente en la que el servidor puede enviar mensajes sin que el cliente los pida. WebSocket queda como mejora al integrar.
Autenticación	De token validado por FastAPI a identificación local	La guía pide no duplicar en Go el servicio de Fundamentos de Seguridad. Se dejó la interfaz Authenticator lista para conectarlo.

Para mantener la trazabilidad actualizamos la matriz, que ahora llega hasta el módulo del código y la prueba automatizada que comprueba cada requisito (ISO/IEC/IEEE, 2018). La Tabla 4 muestra la relación.

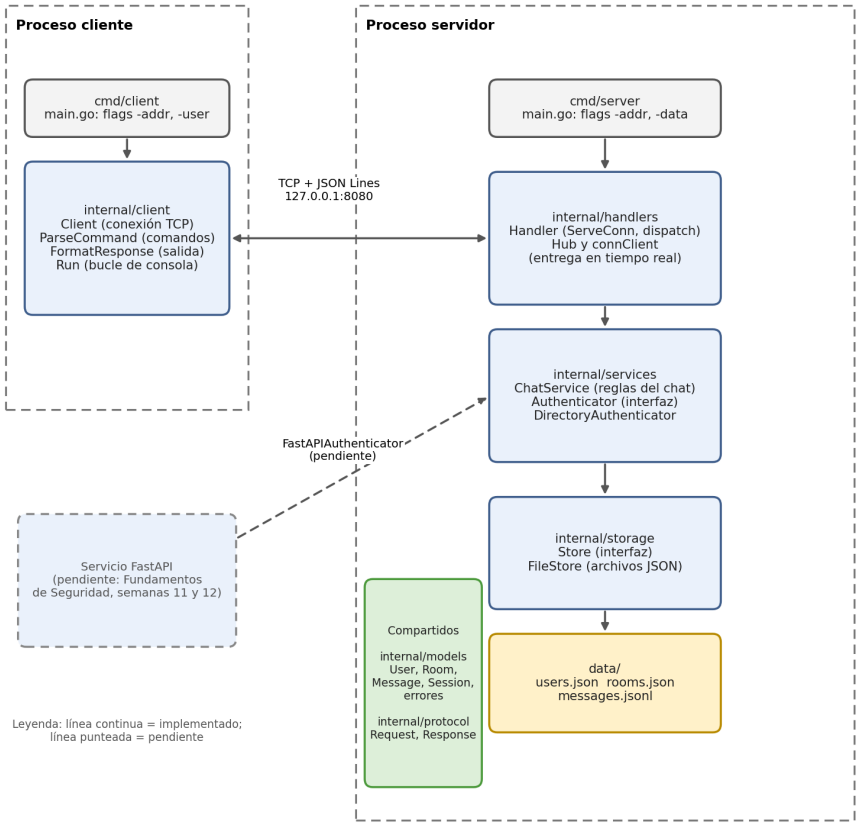
Tabla 4

Trazabilidad actualizada: requisito, módulo y prueba

Requisito	**Módulo del código**	**Prueba automatizada**
RF-02	services: DirectoryAuthenticator y ChatService.Login	TestLoginLogout
RF-03	ChatService.Logout y liberación de sesión en handlers	TestLoginLogout, TestDisconnectReleasesSession
RF-05	ChatService.SendToRoom	TestSendToRoom
RF-06	ChatService.SendDirect	TestSendDirect, TestDirectMessageOnlyReachesParticipants
RF-07	handlers: Hub y deliver	TestRealTimeDelivery
RF-08	ChatService.RoomHistory y DirectHistory	TestHistory, TestHistoryOverNetwork
RF-09	ChatService.JoinRoom y ListRooms	TestJoinRoom, TestListRooms
RF-10	ChatService.OnlineUsers	TestLoginLogout
RNF-06	handlers: respuestas de error	TestValidationErrorsAreReported, TestInternalErrorsDoNotLeakDetails
RNF-08	models: NewRoomMessage y NewDirectMessage	TestMessageConstructors, TestValidateText

5. Diagrama actualizado de componentes y módulos

La Figura 1 muestra cómo se organiza el prototipo. Hay dos procesos independientes, el cliente y el servidor, y dentro del servidor una cadena de capas: los manejadores atienden la red, los servicios aplican las reglas del chat y el almacenamiento lee y escribe los datos. Los paquetes models y protocol los usan ambos procesos (UIDE, 2026b).



Todo el código vive dentro de la carpeta internal, que en Go impide que otros proyectos importen esos paquetes (The Go Authors, s. f.-a). Cada paquete tiene una sola responsabilidad y depende solo de los que están por debajo de él, así que, por ejemplo, el servicio nunca sabe que existe una red y el almacenamiento nunca conoce las reglas del chat (Donovan y Kernighan, 2015). Las líneas punteadas son lo que falta: el servicio FastAPI se conectará a la interfaz Authenticator sin tocar el resto.

6. Comunicación cliente-servidor

El cliente y el servidor son programas distintos que se comunican por TCP, un protocolo que ofrece una conexión confiable y ordenada entre dos extremos (Eddy, 2022). Cada mensaje es un objeto JSON (Bray, 2017) escrito en una sola línea. Esa elección permite que la conexión quede abierta y que el servidor envíe un mensaje al cliente en cualquier momento, que es lo que necesita un chat. La dirección y el puerto se configuran con el parámetro -addr, cuyo valor por defecto es 127.0.0.1:8080. En el diseño de red del proyecto el servidor estará en 10.10.0.194:8080, dentro de la VLAN de servidores de la Sede A (UIDE, 2026a).

El recorrido de una solicitud es el siguiente. El cliente interpreta lo que escribe la persona y envía un Request. El servidor lo lee, lo convierte de JSON, ejecuta el comando en el servicio y contesta con una Response. Si la solicitud es un mensaje, el servicio devuelve además la lista de destinatarios y el servidor entrega el mensaje, como un evento de tipo message, a las conexiones de los que estén conectados. Para atender a varios clientes a la vez, el servidor lanza una goroutine por conexión y protege con candados las estructuras compartidas (Donovan y Kernighan, 2015). La Tabla 5 resume los comandos del protocolo.

Tabla 5

Comandos del protocolo entre cliente y servidor

Comando	**Solicitud (campos)**	**Respuesta o efecto**
login	user	Abre la sesión o devuelve un error si el usuario no existe, está desactivado o ya está conectado
logout, quit	Ninguno	Cierra la sesión (y la conexión, en quit)
rooms	Ninguno	Lista las salas e indica en cuáles es miembro
join	room	Agrega al usuario a la sala
say	room, text	Guarda el mensaje y lo entrega en tiempo real a los miembros conectados

dm	to, text	Guarda el mensaje y lo entrega al destinatario y al remitente
history	room, limit	Devuelve los últimos mensajes de la sala
dmhistory	with, limit	Devuelve los últimos mensajes con esa persona
who	Ninguno	Devuelve los usuarios conectados

Los errores de comunicación se tratan en los dos extremos. Si el servidor no está disponible, el cliente muestra un mensaje que indica la dirección que intentó y termina. Si la conexión se corta a mitad de la sesión, el cliente lo informa y el servidor libera la sesión de esa persona. Si llega un JSON mal formado o una línea demasiado larga, el servidor responde con un error o cierra la conexión, sin caerse. Los fallos internos, como un error al escribir en disco, se registran con detalle en el servidor, pero al cliente solo le llega un mensaje genérico para no exponer rutas ni datos internos (OWASP Foundation, s. f.).

Dejamos dos puntos de contacto con las otras materias. El primero es la interfaz Authenticator, que es por donde se conectará el servicio de autenticación FastAPI en las semanas 11 y 12 (Ramírez, s. f.). El segundo es el registro de eventos, que sigue la misma lógica de Fundamentos de Seguridad: guarda quién hizo qué y con qué resultado, pero nunca el contenido de los mensajes ni credenciales (UIDE, 2026a).

7. Estructuras, constructores y métodos

La Tabla 6 reúne las estructuras principales con su constructor, sus métodos y el concepto de Programación Orientada a Objetos que ilustran. Go no tiene clases, y las estructuras con métodos cumplen ese papel (Donovan y Kernighan, 2015).

Tabla 6

Principales estructuras, constructores y métodos

Paquete	Estructura	Constructor	Métodos principales	Concepto
models	User	NewUser	Username, Role, Active, IsAdmin, SetActive	Encapsulamiento: campos no exportados
models	Room	NewRoom	AddMember, IsMember, Members	Encapsulamiento y copia defensiva
models	Message	NewRoomMessage, NewDirectMessage	IsDirect, BetweenUsers	Validación en el constructor
models	Session	NewSession	User, Username, StartedAt	Composición con *User
storage	Store y FileStore	NewFileStore	LoadUsers, LoadRooms, AppendMessage, Messages	Interfaz y candado
services	ChatService	NewChatService	Login, Logout, SendToRoom, SendDirect, RoomHistory, DirectHistory, JoinRoom	Composición y reglas de negocio
services	Authenticator y DirectoryAuthenticator	NewDirectoryAuthenticator	Authenticate	Interfaz para conectar FastAPI
handlers	Handler, Hub y connClient	NewHandler, NewHub	ServeConn, dispatch, deliver, Send	Goroutines y candados
client	Client	Dial	Send, Listen, Close	Conexión y errores de red

El encapsulamiento se logra con los campos no exportados. Un User solo puede crearse con NewUser, que valida el nombre y el rol, y solo se puede cambiar su estado con SetActive, de modo que ninguna otra capa lo deja en un estado inválido. En Go, la visibilidad se decide por la primera letra del nombre, y los constructores son funciones corrientes que devuelven un valor ya

validado (The Go Authors, s. f.-b). Room.Members devuelve una copia del contenido interno para que quien llame no pueda modificarlo por accidente.

Los métodos que cambian el estado, como SetActive y AddMember, usan receptor por puntero, y los que solo leen, como los de Message, usan receptor por valor (The Go Authors, s. f.-c). La composición aparece en Session, que guarda un puntero a su User, y en ChatService, que se compone de un Store, un Authenticator y los mapas de usuarios, salas y sesiones. Las interfaces se usaron solo donde separan responsabilidades de verdad: Store permite cambiar los archivos por una base de datos, y Authenticator permite conectar el servicio FastAPI.

Los errores se manejan como valores y nunca con panic. Los errores de dominio, como ErrEmptyMessage o ErrNotMember, son variables comparables con errors.Is, y los errores de bajo nivel se envuelven con fmt.Errorf para conservar la causa (Gerrand, 2011). Gracias a eso, el servidor distingue un error que puede mostrarle al usuario de un fallo interno que solo debe registrar.

8. Evidencias de ejecución

Las pruebas automatizadas del proyecto, 25 pruebas principales más sus casos, se ejecutaron con el detector de condiciones de carrera activado y todas pasaron. La salida de go test -race fue la siguiente (Go 1.22.2, Linux).

```
ok      internal/client 1.009s
ok      internal/handlers   1.031s
ok      internal/models 1.009s
ok      internal/services   1.014s
ok      internal/storage    1.015s
```

Para la demostración usamos el script scripts/demo.sh, que levanta el servidor y dos clientes como procesos independientes. La salida del cliente de ariel muestra el envío de un

mensaje, la validación de un mensaje vacío y los errores de usuario y sala inexistentes (UIDE, 2026b).

```
Conectado a 127.0.0.1:18080. Escribe /help para ver los comandos.
OK: sesión iniciada como ariel
Sala actual: general
[19:42:11] [general] ariel: hola equipo, el prototipo ya está corriendo
OK: mensaje 1 enviado a general
ERROR: el mensaje está vacío
[19:42:11] (privado) ariel -> jorge: nos vemos en la demo
OK: mensaje 2 enviado a jorge
ERROR: el usuario no existe
ERROR: la sala no existe
Conectados: ariel, jorge
OK: sesión cerrada
```

La salida del cliente de jorge muestra que recibió los mensajes en tiempo real, consultó el historial y fue rechazado al escribir en una sala de la que no es miembro. Después de unirse con /join, quedó como miembro de la sala.

```
Conectado a 127.0.0.1:18080. Escribe /help para ver los comandos.
OK: sesión iniciada como jorge
[19:42:11] [general] ariel: hola equipo, el prototipo ya está corriendo
[19:42:11] (privado) ariel -> jorge: nos vemos en la demo
Salas: general (miembro), proyectos
Historial (1 mensaje):
  [19:42:11] [general] ariel: hola equipo, el prototipo ya está corriendo
Historial (1 mensaje):
  [19:42:11] (privado) ariel -> jorge: nos vemos en la demo
ERROR: no eres miembro de esa sala
OK: te uniste a la sala proyectos
Conectados: ariel, jorge
OK: sesión cerrada
```

El registro del servidor deja constancia de cada evento con su usuario, origen y resultado, sin el contenido de los mensajes. Las líneas con resultado rechazado son los casos de validación y error, y la última parte muestra a los dos usuarios que no pueden entrar: uno inexistente y otro con la cuenta desactivada (OWASP Foundation, s. f.).

```
19:42:09 servidor escuchando en 127.0.0.1:18080 (datos en /tmp/tmp.cK84LoQfUQ)
19:42:10 evento=conexion usuario=- origen=127.0.0.1:48264 resultado=abierta
19:42:10 evento=conexion usuario=- origen=127.0.0.1:48274 resultado=abierta
19:42:10 evento=login usuario=jorge origen=127.0.0.1:48264 resultado=ok
19:42:10 evento=login usuario=ariel origen=127.0.0.1:48274 resultado=ok
19:42:11 evento=mensaje usuario=ariel origen=127.0.0.1:48274 resultado=ok detalle="sala=g
eneral id=1 entregado_a=2"
19:42:11 evento=say usuario=ariel origen=127.0.0.1:48274 resultado=rechazado detalle="el m
ensaje está vacío"
19:42:11 evento=mensaje usuario=ariel origen=127.0.0.1:48274 resultado=ok detalle="directo
id=2 entregado_a=2"
19:42:11 evento=dm usuario=ariel origen=127.0.0.1:48274 resultado=rechazado detalle="el us
uario no existe"
19:42:11 evento=say usuario=ariel origen=127.0.0.1:48274 resultado=rechazado detalle="la sa
la no existe"
19:42:14 evento=historial usuario=jorge origen=127.0.0.1:48264 resultado=ok detalle="sala=g
eneral"
19:42:14 evento=historial usuario=jorge origen=127.0.0.1:48264 resultado=ok detalle="con=a
riel"
19:42:14 evento=say usuario=jorge origen=127.0.0.1:48264 resultado=rechazado detalle="no
eres miembro de esa sala"
19:42:14 evento=join usuario=jorge origen=127.0.0.1:48264 resultado=ok detalle="sala=proye
ctos"
19:42:15 evento=logout usuario=jorge origen=127.0.0.1:48264 resultado=ok
19:42:15 evento=conexion usuario=- origen=127.0.0.1:48264 resultado=cerrada
19:42:15 evento=logout usuario=ariel origen=127.0.0.1:48274 resultado=ok
19:42:15 evento=conexion usuario=- origen=127.0.0.1:48274 resultado=cerrada
19:42:16 evento=conexion usuario=- origen=127.0.0.1:33770 resultado=abierta
19:42:16 evento=login usuario=- origen=127.0.0.1:33770 resultado=rechazado detalle="usuari
o=\"fantasma\": el usuario no existe"
19:42:16 evento=conexion usuario=- origen=127.0.0.1:33770 resultado=cerrada
19:42:16 evento=conexion usuario=- origen=127.0.0.1:33778 resultado=abierta
19:42:16 evento=login usuario=- origen=127.0.0.1:33778 resultado=rechazado detalle="usuari
o=\"lucia\": la cuenta está desactivada"
19:42:16 evento=conexion usuario=- origen=127.0.0.1:33778 resultado=cerrada
19:42:16 apagando el servidor
```

Los mensajes quedaron guardados en el archivo messages.jsonl, un mensaje por línea, con la hora asignada por el servidor.

```
{"id":1,"from":"ariel","room":"general","text":"hola equipo, el prototipo ya está corriendo","se
nt_at":"2026-11-09T19:42:11.998254749Z"}
{"id":2,"from":"ariel","to":"jorge","text":"nos vemos en la demo","sent_at":"2026-11-09T19:4
2:11.998446205Z"}
```

```
[arielguerrerovelasco@MacBook-Pro-de-Ariel ~ % cd chat-empresarial-go-fastapi/app-go
[arielguerrerovelasco@MacBook-Pro-de-Ariel app-go % go test -race ./...
?       github.com/Cxrnvy/chat-empresarial-go-fastapi/app-go/cmd/client [no test files]
?       github.com/Cxrnvy/chat-empresarial-go-fastapi/app-go/cmd/server [no test files]
ok      github.com/Cxrnvy/chat-empresarial-go-fastapi/app-go/internal/client    1.280s
ok      github.com/Cxrnvy/chat-empresarial-go-fastapi/app-go/internal/handlers  1.517s
ok      github.com/Cxrnvy/chat-empresarial-go-fastapi/app-go/internal/models    1.673s
?       github.com/Cxrnvy/chat-empresarial-go-fastapi/app-go/internal/protocol  [no test files]
ok      github.com/Cxrnvy/chat-empresarial-go-fastapi/app-go/internal/services  1.912s
ok      github.com/Cxrnvy/chat-empresarial-go-fastapi/app-go/internal/storage   2.119s
```

```
arielguerrerovelasco@MacBook-Pro-de-Ariel app-go % bash scripts/demo.sh
== Compilando ==
== Iniciando el servidor en 127.0.0.1:18080 ==
== Clientes (dos procesos independientes) ==
== Cuentas que no pueden entrar ==
ERROR: el usuario no existe
OK: hasta pronto
ERROR: la cuenta esta desactivada
OK: hasta pronto
== Error de conexion (servidor inexistente) ==
error: no se pudo conectar con el servidor en 127.0.0.1:1: dial tcp 127.0.0.1:1: connect: connection refused

=============== SALIDA DEL CLIENTE ariel ===============
Conectado a 127.0.0.1:18080. Escribe /help para ver los comandos.
OK: sesion iniciada como ariel
Sala actual: general
[18:54:13] [general] ariel: hola equipo, el prototipo ya está corriendo
OK: mensaje 1 enviado a general
ERROR: el mensaje está vacio
[18:54:13] (privado) ariel -> jorge: nos vemos en la demo
OK: mensaje 2 enviado a jorge
ERROR: el usuario no existe
ERROR: la sala no existe
Conectados: ariel, jorge
OK: sesion cerrada

=============== SALIDA DEL CLIENTE jorge ===============
Conectado a 127.0.0.1:18080. Escribe /help para ver los comandos.
OK: sesion iniciada como jorge
[18:54:13] [general] ariel: hola equipo, el prototipo ya está corriendo
[18:54:13] (privado) ariel -> jorge: nos vemos en la demo
Salas: general (miembro), proyectos
Historial (1 mensaje):
  [18:54:13] [general] ariel: hola equipo, el prototipo ya está corriendo
Historial (1 mensaje):
  [18:54:13] (privado) ariel -> jorge: nos vemos en la demo
ERROR: no eres miembro de esa sala
OK: te uniste a la sala proyectos
Conectados: ariel, jorge
OK: sesion cerrada

=============== REGISTRO DEL SERVIDOR ===============
2026/11/09 18:54:11 servidor escuchando en 127.0.0.1:18080 (datos en /var/folders/5b/0s0fsv6s70zgkvxqv4t40cl40000gn/T/tmp.XdHhNsOr56)
2026/11/09 18:54:12 evento=conexion usuario=- origen=127.0.0.1:50925 resultado=abierta
2026/11/09 18:54:12 evento=conexion usuario=- origen=127.0.0.1:50926 resultado=abierta
2026/11/09 18:54:12 evento=login usuario=jorge origen=127.0.0.1:50925 resultado=ok
2026/11/09 18:54:12 evento=login usuario=ariel origen=127.0.0.1:50926 resultado=ok
2026/11/09 18:54:13 evento=mensaje usuario=ariel origen=127.0.0.1:50926 resultado=ok detalle="sala=general id=1 entregado_a=2"
2026/11/09 18:54:13 evento=say usuario=ariel origen=127.0.0.1:50926 resultado=rechazado detalle="el mensaje está vacio"
2026/11/09 18:54:13 evento=mensaje usuario=ariel origen=127.0.0.1:50926 resultado=ok detalle="directo id=2 entregado_a=2"
2026/11/09 18:54:13 evento=dm usuario=ariel origen=127.0.0.1:50926 resultado=rechazado detalle="el usuario no existe"
2026/11/09 18:54:13 evento=say usuario=ariel origen=127.0.0.1:50926 resultado=rechazado detalle="la sala no existe"
2026/11/09 18:54:16 evento=historial usuario=jorge origen=127.0.0.1:50925 resultado=ok detalle="sala=general"
2026/11/09 18:54:16 evento=historial usuario=jorge origen=127.0.0.1:50925 resultado=ok detalle="con=ariel"
2026/11/09 18:54:16 evento=say usuario=jorge origen=127.0.0.1:50925 resultado=rechazado detalle="no eres miembro de esa sala"
2026/11/09 18:54:16 evento=join usuario=jorge origen=127.0.0.1:50925 resultado=ok detalle="sala=proyectos"
2026/11/09 18:54:17 evento=logout usuario=ariel origen=127.0.0.1:50926 resultado=ok
2026/11/09 18:54:17 evento=conexion usuario=- origen=127.0.0.1:50926 resultado=cerrada
2026/11/09 18:54:17 evento=logout usuario=jorge origen=127.0.0.1:50925 resultado=ok
2026/11/09 18:54:17 evento=conexion usuario=- origen=127.0.0.1:50925 resultado=cerrada
2026/11/09 18:54:17 evento=conexion usuario=- origen=127.0.0.1:50927 resultado=abierta
2026/11/09 18:54:17 evento=login usuario=- origen=127.0.0.1:50927 resultado=rechazado detalle="usuario=\"fantasma\": el usuario no existe"
```

```
2026/11/09 18:54:17 evento=conexion usuario=- origen=127.0.0.1:50927 resultado=cerrada
2026/11/09 18:54:17 evento=conexion usuario=- origen=127.0.0.1:50928 resultado=abierta
2026/11/09 18:54:17 evento=login usuario=- origen=127.0.0.1:50928 resultado=rechazado detalle="usuario=\"lucia\": la cuenta está desactivada"
2026/11/09 18:54:17 evento=conexion usuario=- origen=127.0.0.1:50928 resultado=cerrada
2026/11/09 18:54:17 apagando el servidor

=============== ARCHIVO DE MENSAJES ===============
{"id":1,"from":"ariel","room":"general","text":"hola equipo, el prototipo ya está corriendo","sent_at":"2026-11-09T18:54:13.659192-05:00"}
{"id":2,"from":"ariel","to":"jorge","text":"nos vemos en la demo","sent_at":"2026-11-09T18:54:13.66026-05:00"}
arielguerrerovelasco@MacBook-Pro-de-Ariel app-go % /rooms
zsh: no such file or directory: /rooms
arielguerrerovelasco@MacBook-Pro-de-Ariel app-go %
```

9. Limitaciones actuales del prototipo

El prototipo solo identifica a los usuarios: no hay contraseñas ni tokens, así que cualquier persona que conozca un usuario activo puede entrar con su nombre. Tampoco hay cifrado, por lo que el tráfico y el archivo de mensajes están en texto plano, aunque el archivo se crea con permisos restringidos. Estas carencias son las que deben cubrir el servicio FastAPI y TLS, que

corresponden al diseño de seguridad de la materia de Fundamentos de Seguridad (UIDE, 2026a). Mientras tanto, el prototipo debe usarse solo en un entorno local o de laboratorio, como permite la guía (UIDE, 2026b).

También hay límites funcionales. Las membresías que se agregan con /join viven solo en memoria y se pierden al reiniciar el servidor. El historial lee todo el archivo en cada consulta y devuelve como máximo los últimos 100 mensajes. No hay vencimiento de sesiones por inactividad, reconexión automática ni estadísticas, y el prototipo solo se probó en Linux. Todo esto está anotado como pendiente en la Tabla 2 y se retomará en las siguientes fases (UIDE, 2026b).

10. Enlace al repositorio Git

El código fuente, el README con las instrucciones de ejecución y los datos de prueba están en https://github.com/Cxrnvy/chat-empresarial-go-fastapi , dentro de la carpeta app-go (UIDE, 2026b).

11. Enlace al video demostrativo

https://drive.google.com/drive/folders/1ShSUIG8IQVVvxMQF60Tu3Zb-E3wm589q?hl=es

Declaración de uso de IA

Usamos Claude, un modelo de lenguaje de Anthropic (2026), como apoyo para estructurar el código del prototipo, redactar las pruebas y escribir este documento y el README. El equipo revisó el contenido, ejecutó el código y asume el resultado.

Referencias

- Anthropic. (2026). *Claude Sonnet 5.5* [Modelo de lenguaje de gran tamaño]. https://claude.ai
- Bray, T. (Ed.). (2017). *The JavaScript Object Notation (JSON) data interchange format* (RFC 8259). Internet Engineering Task Force. https://doi.org/10.17487/RFC8259
- Donovan, A. A. A., y Kernighan, B. W. (2015). *The Go programming language*. Addison-Wesley.
- Eddy, W. (Ed.). (2022). *Transmission control protocol (TCP)* (RFC 9293). Internet Engineering Task Force. https://doi.org/10.17487/RFC9293
- Gerrand, A. (2011, 12 de julio). *Error handling and Go*. The Go Blog. https://go.dev/blog/error-handling-and-go
- ISO/IEC/IEEE. (2018). *Systems and software engineering: Life cycle processes: Requirements engineering* (ISO/IEC/IEEE 29148:2018).
- OWASP Foundation. (s. f.). *Input validation cheat sheet*. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html
- Ramírez, S. (s. f.). *FastAPI*. https://fastapi.tiangolo.com/
- The Go Authors. (s. f.-a). *Command go: Internal directories*. https://pkg.go.dev/cmd/go#hdr-Internal_Directories
- The Go Authors. (s. f.-b). *Effective Go*. https://go.dev/doc/effective_go
- The Go Authors. (s. f.-c). *The Go programming language specification*. https://go.dev/ref/spec
- Universidad Internacional del Ecuador. (2026a). *Proyecto Integrador – Tercer nivel: Desarrollo de una plataforma empresarial distribuida integrando redes, desarrollo de software y seguridad informática* [Documento del curso].
- Universidad Internacional del Ecuador. (2026b). *Segundo entregable del proyecto integrador: Prototipo funcional de la aplicación cliente-servidor en Go* [Instrucciones de la actividad]. Plataforma Canvas de la UIDE.